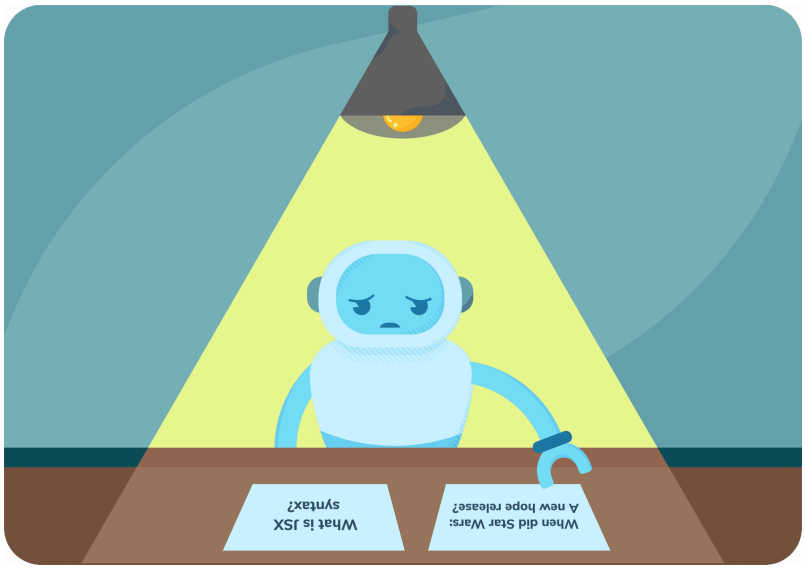
Completing Incomplete Content and Factual Responses

Explore how to complete unfinished sentences and generate accurate factual answers using OpenAI's completions endpoint. Learn to adjust parameters like temperature for focused or creative text generation, and apply these methods for practical writing and content tasks.

Overview

The chat completions endpoint can also be used to complete unattended (incomplete) sentences. For example, if we leave a sentence half-written, the chat completions endpoint can complete the sentence provided that it has the appropriate context. In this case, the value of `temperature` is usually kept low so that the completion does not disregard the previous context.



Ask

Factual responses

We input the incomplete content and request parameters into the OpenAI API's model to complete that content.

Completions: complete incomplete content

Content completion

Here is an example of an incomplete sentence that we can provide to the API for content completion:

```
Although the tiger is a solitary beast and\n
```

Example

Let's try the above example in the code widget below:

Please provide values for the following:

SECRET_KEY Not Specified...

Ask

Edit Import Values from JSON

main.py Python

```python
# Finding completion of unattended sentence

from openai import OpenAI
client = OpenAI(api_key="==SECRET_KEY==")

response = client.chat.completions.create(
  model="gpt-3.5-turbo",
  messages=[
    {"role": "system", "content": "Find completion of unattended sentence
    {"role": "user", "content": "Although the tiger is a solitary beast a
  ],
)
print(response.choices[0].message.content)
```

Run

Finding completion of unattended sentence

If we want to generate a focused completion of the unattended sentence, we use a lower `temperature` value. Otherwise, if we want a more creative and abstract completion, we use a higher `temperature` value. In the code above, we use a low value of the `temperature` parameter. The value of `top_p` indicates that we include all the tokens in the results. The default values of `presence_penalty` and `frequency_penalty` are used. We can experiment with these parameters by changing these values in multiple runs and trying different `engines` to see their capabilities.

Factual responses

The API has a good grasp of the data on which it is trained and can generate answers to hypothetical questions. If we want the API to reply with less va answers, we should tell it how to respond when it does not know the answer.

Ask

Here is an example of the prompt:

```
Q: Who is Batman?
A: Batman is a fictional comic book character.

Q: What is torsalplexity?
A: ?

Q: What is Devz9?
A: ?

Q: Who is George Lucas?
A: George Lucas is an American film director and producer famous for creating
...
<more questions-answers, if needed>
...
Q: What is the meaning of xyzz?
A:
```

In the example above, we have set ? as an answer if it is unknown. The model will follow the same pattern for unknown answers.

Example

Let's run the above example in the code widget below:

Please provide values for the following:

SECRET_KEY Not Specified...

Edit Import Values from JSON

main.py Python

```python
# Finding factual responses

from openai import OpenAI
client = OpenAI(api_key="<<SECRET_KEY>>")

response = client.chat.completions.create(
  model="gpt-3.5-turbo",
  messages=[
```

Ask

```
        {"role": "system", "content": "You give factual answers. If there is
        {"role": "user", "content": ("Q: Who is Batman?\n"+
                "A: Batman is a fictional comic book character.\n"+
                "Q: What is torsalplexity?\n"+
                "A: ?\n"+
```

Run

Finding factual responses

The response for the `prompt_text` above will be `?` because the model does not know the meaning of `xyzz`.

Now, replace the last question with the following text and observe the behavior:

```
Q: How many countries are in the world?
```

The model will return the actual answer because it is known information for the model. We use the lowest `temperature` value in the code above because the answer should be straightforward and factual, with no randomness or other flourishes.

Practical use case

Some practical use cases of OpenAI's content completion and factual response capabilities include:

- **Creative writing prompt**: A writer experiences writer's block while working on a story. They have an incomplete sentence or paragraph but struggle to continue the narrative. The writer can input the incomplete sentence or paragraph into the chat completions endpoint, providing necessary context about the story's plot, characters, and themes.
- **Content creation for social media posts**: A social media manager s drafting a post but gets interrupted and cannot finish it. They want to complete the post later with the same style and voice. The social media

Ask

manager can input the incomplete post into the chat completions endpoint, ensuring to include relevant hashtags or mentions for context. The model generates the remaining text by setting the temperature appropriately and completing the post per the brand's tone and style.

Ask